Homework Sets

15–20 minutes a week: Core, Challenge and Creative

Tip: How homework works

Do the **Core** task first (about 10 minutes). **Challenge** and **Creative** are optional extras for when you want more. Finish with the self-check list. If you get stuck for more than 5 minutes, write down what you tried and bring the question next week: that is a great result too.

Contents

Week 1: Welcome Back, Python 2
Week 2: Functions That Give Back 4
Week 3: Lists: Collections of Things 6
Week 4: While Loops and Game Loops 8
Week 5: Dictionaries and Secret Codes 10
Week 6: Review and Checkpoint 12
Week 7: Files: Saving Your Stuff 14
Week 8: Testing: Prove It Works 16
Week 9: Event-Driven Games 18
Week 10: Project Studio 1 20
Week 11: Project Studio 2 22
Week 12: Showcase and Celebration 24

Week 1
Welcome Back, Python
Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Type detective (paper, 3 minutes)

Write the type of each value: **int**, **float**, **str** or **bool**.

`42`		`"42"`	
`4.2`		`False`	
`"False"`		`-8`	

★ Core (everyone): Dragon Door (Thonny, 10 minutes)

Open `exercises/week01/hw-dragon-door/starter.py`. The dragon door opens if:

- you have the golden key **and** you are level 3 or more, **or**
- you are level 10 or more (legends do not need keys).

1. TODO 1: change the `if` line in `door_opens()`. Brackets help: `(has_key and level >= 3) or level >= 10`
2. TODO 2: make the welcome message an f-string that says the hero name.
3. Run it three times and check what happened:

   key yes, level 5: opens ☐ shut ☐
   key yes, level 2: opens ☐ shut ☐
   key no, level 12: opens ☐ shut ☐

4. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Predict, then check

Write what each line prints. Then type it into the Thonny Shell to check.

```
print(3 > 2 and 5 > 9)
print(3 > 2 or 5 > 9)
print(not 3 > 2)
print("7" + "7")
```

```
print(int("7") + 7)
```

Creative (make it yours): Design a hero

Invent a hero for a game (a made-up name only). Draw them in the box and write a hero card with at least one value of each type: a str, an int, a float and a bool. Next week you can type it into the Hero Card program.

Self-check: tick each box when it is true

- ☐ I can name the four types and give an example of each.
- ☐ I know that `input()` gives back text, and how `int()` fixes that.
- ☐ My Dragon Door opens for the right heroes and stays shut for the others.
- ☐ I used a made-up hero name, not my real name.

Week 2 Level 2: Intermediate

Functions That Give Back

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): What comes back? (paper, 3 minutes)

Write what each call gives back. Write `None` if it gives back nothing.

```
def add_three(n):
    return n + 3

def shout(word):
    print(word + "!")
```

`add_three(4)` gives back _____

`add_three(add_three(1))` gives back _____

`shout("hi")` gives back _____

★ Core (everyone): Potion Shop Calculator (Thonny, 10 minutes)

Open `exercises/week02/hw-shop-calculator/starter.py`. You have 50 coins and potions cost 12 coins each.

1. TODO 1: `total_cost()` returns price times quantity.
2. TODO 2: `can_afford()` returns True or False.
3. TODO 3: write a docstring for `coins_left()`.
4. TODO 4: finish `coins_left()`. You can call `can_afford()` inside it!
5. Run it. Buying 3 potions leaves _____ coins. Buying 5 potions leaves _____ coins.
6. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Sale day

Add a function `discount(cost)` that returns the cost minus 5 if the cost is 40 or more, and the same cost otherwise. Use it in the main program before checking `can_afford()`. Write a docstring for it. What do 5 potions cost on sale day? _____

Creative (make it yours): Invent a function machine

Invent a secret rule for a function machine (for example “add the two numbers, then double”). Try it on someone at home: they hand you values, you hand back answers, they guess the rule. Then write your rule as a Python function with a docstring:

Self-check: tick each box when it is true

- ☐ I can explain the difference between `print` and `return`.
- ☐ All three shop functions return their answer (no `None` in my output).
- ☐ Every function in my homework has a docstring.
- ☐ `check_my_work.py` shows `[PASS]` for my shop calculator.

Week 3 Level 2: Intermediate

Lists: Collections of Things

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Index practice (paper, 3 minutes)

```
1 team = ["Ada", "Kai", "Mei", "Omar", "Zoe"]
```

`team[0]` = _____ `team[3]` = _____

`team[-1]` = _____ `len(team)` = _____

Does `team[5]` work? Why or why not?

★ Core (everyone): Loot Bag (Thonny, 10 minutes)

Open `exercises/week03/hw-loot-bag/starter.py`. You open a treasure chest several times and keep the loot in a list.

1. Press **Run**. The bag is empty!
2. TODO 1: in `fill_bag()`, repeat `how_many` times and append `open_chest(chest)` to the bag.
3. TODO 2: in `count_item()`, add 1 to `count` each time a thing in the bag matches the item.
4. Run it three times. Is the loot the same each time? Why not? _____
5. Open `check_my_work.py` and press **Run**.

★★ Challenge (want more?): Rare treasure

`"gold coin"` is in CHEST three times, so it is three times as likely as the ruby. Add a super-rare item that appears only once, and make `"old boot"` more common. Then open the chest 20 times: how many of your rare item did you get? _____

Creative (make it yours): Design a palette

Choose 4 colors from the Okabe-Ito list for a game or poster you would like to make. Write the list as Python, then sketch your design. Add a label or shape to every colored part, so someone who sees colors differently can still understand it.

Self-check: tick each box when it is true

- ☐ I know the first item of a list has index `0`, and the last has index `len - 1` (or `-1`).
- ☐ My `fill_bag()` uses `append` inside a loop.
- ☐ `check_my_work.py` shows `[PASS]` for my loot bag.
- ☐ My palette design uses labels or shapes, not only colors.

Week 4 Level 2: Intermediate

While Loops and Game Loops

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Trace it first (paper, 4 minutes)

Before you code, follow `hits_needed(30, 7)` by hand. Each time round the loop, 7 is taken away from `health` and 1 is added to `hits`. Keep going until `health` is `0` or less.

round	health	hits
start	30	0
1	23	1
2		
3		
4		
5		

So `hits_needed(30, 7)` gives back: _____

★ Core (everyone): Slime Battle (Thonny, 10 minutes)

Open `exercises/week04/hw-slime-battle/starter.py`.

1. TODO 1: finish `hits_needed()` with a `while` loop: while `health` is more than `0`, take away `damage` and add 1 to `hits`.
2. TODO 2: make `ask_yes_no()` keep asking until the answer is `yes` or `no`.
3. Run it. Type `maybe` first. Does it ask again? ☐
4. Open `check_my_work.py` and press **Run**.

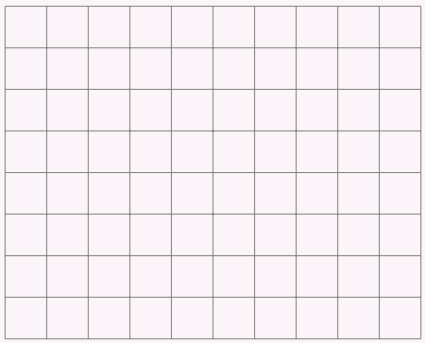
Stuck in a loop? Press the red **Stop** button and check that `health` really gets smaller inside the loop.

★★ Challenge (want more?): Count up to a target

Write a function `days_to_save(goal, per_day)` that uses a `while` loop to count how many days of saving `per_day` coins it takes to reach `goal` coins. For example `days_to_save(50, 12)` is 5. Add it to the bottom of your Slime Battle file (above `if __name__ == "__main__":`) and try it in the Shell.

Creative (make it yours): Pixel sprite design

Design a pixel sprite for a game: a slime, a pet, a spaceship or a potion. Color the squares and write the letter code for each row underneath (K, O, Y, B, S, G, R, P or . for empty). Next week you can type it into the Pixel Art program.



Self-check: tick each box when it is true

- ☐ I can explain what makes a `while` loop stop.
- ☐ I know what to do when a program is stuck in an infinite loop.
- ☐ My Slime Battle asks again when I type something that is not yes or no.
- ☐ All the checks in `check_my_work.py` pass (or I wrote down my question for next week).

Week 5

Dictionaries and Secret Codes

Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Keys and values (paper, 3 minutes)

```
pet = {"name": "Mochi", "legs": 4, "can_fly": False}
```

1. What does `pet["legs"]` give back? _____
2. Write a line that adds the key `"color"` with the value `"purple"`:

   _____
3. What happens with `pet["Legs"]`? _____

★ Core (everyone): Word Counter (Thonny, 10 minutes)

Open `exercises/week05/hw-word-counter/starter.py`. It should count how many times each word appears, using a dictionary.

1. Press **Run**. Every word says 1. That is a bug!
2. TODO 1: if the word is already in `counts`, add 1 to its count. Otherwise start it at 1.
3. TODO 2: finish `most_common()` with `for word, count in counts.items():`.
4. Run it. Which word is the most common? _____
5. Open `check_my_work.py` and press **Run**.

Two new string tools are used: `text.lower()` makes all letters small, and `text.split()` cuts the text into a list of words.

★★ Challenge (want more?): Crack the code

This message was made with a Caesar cipher, but nobody told you the shift. Use your paper wheel (or your Caesar program from class) to try shifts until it makes sense. Tip: try the short word first.

`lw lv vdih wr sodb`

The shift was _____. The message is: _____

Creative (make it yours): Invent your own code

Make up your own secret alphabet: draw a symbol for every letter (a star for a, a wave for b ...). Write it as a key table, then write a short message for someone at home and see if they can decode it with your key.

Self-check: tick each box when it is true

- ☐ I can explain what a key and a value are.
- ☐ I know what a `KeyError` means and how to avoid it.
- ☐ My word counter counts repeated words correctly.
- ☐ I know that real passwords stay private and never go into a program.

Week 6
Review and Checkpoint
Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Pixel Pet bug hunt (Thonny, 15 minutes)

Open `exercises/week06/hw-bug-pixel-pet/starter.py`. It is a virtual pet stored in a dictionary, and it has **three** bugs. The rules: `feed()` takes 3 from hunger (never below `0`); `play()` adds 2 to happiness and 1 to hunger; `mood()` says `"grumpy"` if hunger is 8 or more **or** happiness is 2 or less, otherwise `"happy"`.

1. Press **Run**. Read the traceback bottom-up and fix bug 1.
2. Run again and look at the printed dictionary. Is there a key that should not be there? That is bug 2.
3. Bug 3 does not crash. Explain `mood()` out loud (rubber duck!) and compare it with the rules.
4. Open `check_my_work.py` and press **Run**.

Bug	Error name (or “no crash”)	Line	What I changed
1			
2			
3			

★★ Challenge (want more?): Trace it

Fill in a trace table on scrap paper and write what this prints:

```
coins = 1
days = 0
while coins < 20:
    coins = coins * 2
    days = days + 1
print(days, coins)
```

It prints: _____

Creative (make it yours): Bug comic

Draw a 3-panel comic about a bug and the detective who finds it. Panel 1: the program breaks. Panel 2: the detective uses one of the four tools. Panel 3: fixed! Give your bug a name.

Self-check: tick each box when it is true

- ☐ I read tracebacks from the bottom up.
- ☐ I used at least two different detective tools this week.
- ☐ I wrote a bug report for each Pixel Pet bug.
- ☐ When I got stuck, I tried one thing at a time instead of changing everything.

Week 7 Level 2: Intermediate

Files: Saving Your Stuff

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Game Idea Notebook (about 10 minutes)

Folder: `exercises/week07/hw-idea-notebook`. This program keeps a list of your game and art ideas in a file called `game_ideas.txt`. The menu already works; the file part does not.

1. Open `starter.py` and press **Run**. Type `a`, an idea, then `l`. Is the idea listed? (Not yet!)
2. TODO 1: in `add_idea()`, open the file with `"a"` and write the idea plus `"\n"`.
3. TODO 2: in `load_ideas()`, read every line, `strip()` it and keep the ones that are not empty. Catch `FileNotFoundError` and make an empty file, just like in class.
4. Add three ideas, quit with `q`, run the program again and type `l`. They are still there!
5. Open `check_my_work.py` and press **Run**.

Write your favorite idea here. It could become your final project!

_____

★★ Challenge (want more?): Mode detective

Predict what is in `test.txt` after each program runs (start with no file each time). Then try it in Thonny.

Program A

```
for word in ["red", "blue"]:
    with open("test.txt", "w") as file:
        file.write(word + "\n")
```

Program B

```
for word in ["red", "blue"]:
    with open("test.txt", "a") as file:
        file.write(word + "\n")
```

Program A gives: _____ Program B gives: _____

Then add `d` = delete all ideas to your notebook (hint: which mode empties a file?).

Creative (make it yours): Make it yours

Pick one:

- Draw a new picture for `my_art.txt` (Exercise 7.3) on squared paper first: mark each dot's `x`, `y`, size and color. Then type the lines in and run it.
- Make a themed word list for Word Guess: `space.txt`, `animals.txt` or `food.txt`, and change `WORD_FILE` to use it.

Self-check: tick each box when it is true

- ☐ My ideas are still there after I close and run the program again.
- ☐ I can explain the difference between `"w"` and `"a"`.
- ☐ My program does not crash when the file is missing.
- ☐ My files hold ideas, words or pictures only: no real names, addresses or passwords.

Week 8 Level 2: Intermediate

Testing: Prove It Works

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Test the Cipher (about 10 minutes)

Folder: `exercises/week08/hw-test-the-cipher`. `encode()` in `cipher_tools.py` is the Caesar cipher from week 5, and it is correct. Your job: write tests so nobody can ever break it without you noticing. Keep your paper cipher wheel nearby.

1. Read `cipher_tools.py`. Then open `starter.py` (your test file) and press **Run**.
2. TODO 1: work out `encode("xyz", 3)` with the wheel and write a test for it: _____
3. TODO 2: test a message with a space and a `!` in it. My message and its secret version:

   _____
4. TODO 3: one more test of your own (ideas: shift `0`, or an empty message `""`).
5. Press **Run**: all PASSED? Then run `check_my_work.py`. It tries your tests on three broken ciphers.

★★ Challenge (want more?): Spot the edge

For each function, write one **normal** case and one **edge** case (no computer needed).

Function	Normal case	Edge case
`count_vowels(word)`: counts a, e, i, o, u		
`lives_left(lives, hits)`: never below `0`		
`last_item(items)`: the last item		

Creative (make it yours): Test something of yours

Pick a function you wrote in an earlier week (for example from your Monster Stats, Word Guess or Idea Notebook). In the same folder, make a new file called `my_tests.py`. Import your function (`from starter import my_function`), write two tests (one normal, one edge), and copy the last three lines of this week's `starter.py` to the bottom so that **Run** starts pytest. This is exactly how your final project's tests will work. Or: draw a comic strip of a "crash-test dummy" testing a game's strangest corner.

Self-check: tick each box when it is true

- ☐ My test names all start with `test_` and each has an `assert`.
- ☐ All my tests say PASSED on the correct `encode()`.
- ☐ `check_my_work.py` says my tests catch all three broken ciphers.
- ☐ I can explain the difference between a normal case and an edge case.

Week 9 Level 2: Intermediate

Event-Driven Games

15–20 minutes. Do both Core parts; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Part 1: Stamp Studio (about 10 minutes)

Folder: `exercises/week09/hw-stamp-studio`. An art program you play with the keyboard. The current shape and color are kept in a dictionary called `studio`.

1. Open `starter.py`, press **Run** and **click the window**. Arrows move, `s` stamps. `p` and `n` do nothing yet.
2. TODO 1: `next_index(index, length)` returns the next position and goes back to `0` after the last one. Fill in: `next_index(2, 4)` = _____ `next_index(3, 4)` = _____
3. TODO 2: write `next_color()`, a twin of `next_shape()`.
4. TODO 3: connect `"p"` and `"n"` with `screen.onkey(...)`. No brackets after the function name!
5. Make a picture, then run `check_my_work.py`.

★ Core (everyone): Part 2: Project pitch, milestone M1 (about 8 minutes)

Take out your **Project Brief**. Fill in **Planning page 1: My pitch**. If you work with a partner, do it together (in person, or bring your ideas next week). Bring the Brief to the next session: we start by checking milestone M1.

- I chose: ☐ Option A: Arcade Game ☐ Option B: Art Studio
- The keys or buttons my program will use: _____

★★ Challenge (want more?): Plan it, upgrade it

- Start **Planning page 2** of the Project Brief: write the pseudocode or draw the flowchart of your project's main loop.
- Or add a key to Stamp Studio that makes the stamps bigger and smaller with `stamper.shapesize(...)`. Keep the size in the `studio` dictionary.

Creative (make it yours): Draw your game screen

On paper, draw what your project will look like when it is running. Label every part: the player or brush, the things that move, the score or menu, and the keys. Use shapes and labels, not only colors, so everyone can tell the parts apart.

Self-check: tick each box when it is true

- ☐ In Stamp Studio, p and n go round all the shapes and colors and start again.
- ☐ I wrote `onkey(next_shape, "p")` with no brackets after `next_shape`.
- ☐ My pitch page is filled in, and I know which option I chose.
- ☐ If I want to use a picture or sound, I know it needs an open license and a credit.

Week 10
Project Studio 1
Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

Tip: No computer at home this week?

That is fine. Do the paper parts (the dev log, the Challenge plan and the Creative sketch) and bring them next week. Your project files stay in your team folder; never email or upload them.

Core (everyone): Finish your own sticky notes (about 10 minutes)

1. Open your team's project in Thonny. Pick **one** sticky note from your TO DO column that is yours.
2. Build it in small steps. Press F5 after each step. Save with Cmd+S often.
3. If you get a red error, read the last line of the traceback and the line number it points to. Stuck for 5 minutes? Write down what you tried and stop: that is useful too.
4. Write a short **dev log** (developer diary) below.

What I worked on:

What broke, and how I fixed it (or what I tried):

What I will do next:

Challenge (want more?): Tests on paper, before the code

Next week you will write pytest tests. Get ahead: choose one of your functions that `returns` something and write three examples of what goes in and what should come out. Make at least one an **edge case** (a value at the very limit, like the edge of the screen or a score of 0).

Function and input	Expected answer	Normal or edge?
`next_x(0, 1)`	`20`	normal

Creative (make it yours): Title screen sketch

Design the title screen for your project on squared paper: its name, a picture, and one line that explains the keys ("← → to move, space to start"). Use colors from the Okabe–Ito palette (orange, sky blue, bluish green, yellow, blue, vermillion, reddish purple) and make sure nothing depends on color alone. Use your nickname, not your real name.

Self-check: tick each box when it is true

- ☐ I worked on one of my own tasks and saved my work.
- ☐ My program still runs (or I wrote down exactly where it breaks).
- ☐ I wrote my dev log: what I did, what broke, what is next.
- ☐ Anything I copied or remixed has a credit comment.

Week 11
Project Studio 2
Level 2: Intermediate

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Three passing tests and one fix (about 10 minutes)

1. Open your project folder in Thonny. Make sure you have a file called `test_mygame.py` (or `test_` plus your game's name) with **at least 3 tests**.
2. Run your game once with F5, close its window, then **Tools > Open system shell...** and type `python -m pytest -v` (or `python3 -m pytest -v`).
3. Copy the last line of the output here (for example "3 passed in 0.05s"):

   _____

4. Pick the top MUST line from your playtest bug list and fix it. What did you fix?

   _____

★★ Challenge (want more?): Test first

Find a bug or a missing feature in a logic function. Write the test **first** and run it: it should say FAILED. Then change your code until it says PASSED. Write your test here:

Why is it useful to see the test fail first?

_____

Creative (make it yours): How to play

Add a "how to play" message to your project's screen (with `write` and a font size of 18 or more), or write it on a card to place next to the computer at the showcase. Include: the goal, the keys, and how the game ends (or, for an Art Studio, how to draw, save and load). Test it on a family member: can they start without asking you anything?

Self-check: tick each box when it is true

- ☐ At least 3 of my tests say PASSED.
- ☐ My project saves something to a file and loads it again.
- ☐ I fixed at least one bug from the playtest.
- ☐ I checked the accessibility checklist in workbook chapter 11 (colors, shapes, text size, keys explained, no flashing).
- ☐ High-score names and saved files use nicknames only.

Week 12 Level 2: Intermediate

Showcase and Celebration

15–20 minutes for the reflection. The menu is for the holidays: pick whatever you like.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): My Level 2 reflection (about 10 minutes)

1. The part of my project I am proudest of is...
2. The hardest bug I met in Level 2 was... and I fixed it by...
3. One piece of feedback from the gallery walk that I agree with:
4. Something I learned about working with a partner:
5. In Level 3 I am most curious about (git save points, classes, Maze Muncher, something else?):

★★ Challenge (want more?): Version 2 plan

Plan the next version of your project. Write three new features. For each one: the function you would write, what goes in, what comes out, and one test for it.

Creative (make it yours): Holiday “keep coding” menu

Pick one, two, or none: holidays are for resting too. Everything works offline, with no accounts.

- **Level up:** add one wish from your feedback cards to your project.

- **Art jam:** make a generative art picture with a palette list, `random` and a function. Write down your favorite seed.
- **Secret messages:** use your Caesar cipher program to send a coded note to someone at home.
- **Teach it:** show a family member how to play your game, then how one function works.
- **Remix a classic:** turn a past exercise (pixel art, Etch-a-sketch, word guess) into something new. Add a credit comment.
- **Unplugged:** design a board game whose rules are a game loop: what happens every turn?

What I chose and what I made:

Self-check: tick each box when it is true

- ☐ I answered all five reflection questions.
- ☐ I kept my certificate and my workbook somewhere safe.
- ☐ If I shared my project, it was with family and showed only my nickname.
- ☐ I took breaks from the screen (stretch, and look at something about 6 meters away for 20 seconds).